

AAS Leathers

Technical Documentation

Architecture . Frontend . Backend . Database . Deployment

A custom luxury e-commerce platform

Next.js + Express + MongoDB

Contents

1. System Overview
2. Architecture
3. Technology Stack
4. Repository Structure
5. The Frontend
6. The Backend
7. Database Structure
8. Authentication & Security
9. Key Workflows
10. The Admin Panel
11. API Reference
12. Running It Locally
13. Putting It Online
14. Environment Variables
15. Known Limitations & Roadmap

Section 1

System Overview

AAS Leathers is a custom luxury e-commerce platform for a leather-goods brand, built entirely from scratch - no Shopify, WooCommerce, or store templates. It is a complete storefront and order-management system: customers browse an editorial catalogue, place orders as guests or account holders, track those orders from workshop to doorstep, and download invoices; staff manage the catalogue, inventory and orders through a private admin panel.

The system is built as **three independent pieces** that talk to each other over HTTPS. Keeping them separate means each can be hosted, scaled and updated on its own, and the public website stays fast and cheap to serve because it is just static files.

Design intent

The storefront is deliberately calm and editorial - large typography, warm neutral colours, generous whitespace and slow, purposeful motion. The engineering mirrors that restraint: a small number of well-tested modules rather than sprawling boilerplate.

Section 2

Architecture

Three tiers, each deployed independently:

Tier	What it is	Where it runs
Frontend (web)	Next.js storefront + admin, exported as static HTML/CSS/JS	Any static host (Hostinger public_html)
Backend (server)	Express REST API - auth, orders, catalogue, uploads, invoices, email	Node.js app on an api. subdomain
Database	MongoDB - users, products, orders	MongoDB Atlas (managed cloud)

How a request flows

A visitor loads the static site from your domain. As they browse, the page calls the API for live data (the product catalogue, their cart's availability, their orders). The API validates every request, reads and writes the MongoDB database, and returns JSON. The browser never talks to the database directly.

```
Browser --HTTPS--> Static Frontend (yourdomain.com)
Browser --HTTPS--> Express API --> MongoDB (api.yourdomain.com -> Atlas)
```

Zero-config API address

The frontend resolves its API automatically: in the browser it calls `api.<the-current-domain>`. One static build therefore works on any domain with no rebuild - and because the API is a sub-domain of the same site, the secure login cookie is shared correctly.

Section 3

Technology Stack

Frontend

Tool	Role
Next.js 16 (App Router)	React framework; static export for cheap, fast hosting
React 19 + TypeScript	UI components with strong typing
Tailwind CSS v4	Utility-first styling; design tokens in globals.css
Framer Motion	Animation - reveals, page motion, micro-interactions
Lenis	Smooth, weighted scrolling
Zustand	Lightweight state (cart, wishlist, auth, UI) with persistence

Backend

Tool	Role
Node.js + Express 4	HTTP server and routing
Mongoose 8	MongoDB models, schemas and validation
jsonwebtoken + bcryptjs	JWT sessions and password hashing
multer	Product-image file uploads
pdfkit	On-the-fly PDF invoices
nodemailer	Transactional email (order confirmations)
helmet, cors, express-rate-limit	Security headers, cross-origin rules, rate limiting

Database

MongoDB. In production, a managed **MongoDB Atlas** cluster. In development, an in-memory MongoDB starts automatically (no install needed) and re-seeds the catalogue on every launch - so any developer can run the whole stack instantly.

Section 4

Repository Structure

Two top-level folders - web/ (frontend) and server/ (backend) - plus deployment helpers.

```
AAS leathers/
web/ Next.js storefront + admin
src/app/ Pages (routes)
page.tsx Home (cinematic story)
shop/ collections/ Catalogue browsing
products/[slug]/ Product detail
account/ login/ ... Customer area + auth
admin/ Staff dashboard, orders, products
privacy-policy/ ... Legal & compliance pages
src/components/ Header, footer, cart drawer, search,
product card, motion, admin forms
src/lib/ api.ts, stores, data, types, use-catalog

server/ Express API
src/app.js App assembly (middleware, routes)
src/server.js Boot: DB connect, seed, listen
src/config/ env.js, db.js
src/models/ user.js, product.js, order.js
src/routes/ auth, products, orders, stats, upload
src/middleware/ auth (RBAC), error handling
src/services/ invoice (PDF), mailer (email)
src/utils/ tokens, api-error
uploads/ Uploaded product images (runtime)

deploy-kit/ Pre-packaged zips + checklist
DEPLOY.md Full deployment guide
```

Section 5

The Frontend

Static export

The site is built with output : "export", which turns the whole storefront into plain HTML, CSS and JavaScript files. There is no Node server required to serve the website - it can live in any web host's public folder, load instantly, and cost almost nothing to run.

Live data on a static site

A static site is normally frozen at build time. To keep it current, the shop grid and the home page's featured section use a small hook, `useCatalog()`. It ships with the catalogue baked in (so the page renders instantly and is never empty), then fetches the live catalogue from the API on load and upgrades - new products, price and 'featured' changes appear without a rebuild. If the API is briefly unreachable, the baked-in catalogue simply remains.

State management

Client state lives in four small Zustand stores:

- **Cart** - items and quantities, saved to the browser so it survives refreshes.
- **Wishlist** - saved pieces, also persisted locally.
- **UI** - whether the cart drawer, search overlay or mobile menu is open.
- **Auth** - the current user and the in-memory access token.

Talking to the API

All API calls go through `src/lib/api.ts`. The short-lived access token is held in memory; the long-lived refresh token is a secure cookie the browser sends automatically. If a call returns 401 (token expired), the client silently refreshes the token once and retries - so the user is never unexpectedly logged out. It also provides helpers for file uploads and for downloading invoice PDFs.

Motion & feel

Framer Motion drives the reveals and micro-interactions; Lenis adds smooth, weighted scrolling. Reusable `Reveal` and `RevealLines` components animate content as it enters view. Everything respects the operating system's 'reduce motion' setting.

Section 6

The Backend

A focused Express API. `app.js` assembles the middleware and routes; `server.js` connects the database, seeds the catalogue and admin account if empty, and starts listening.

Request pipeline

Every request passes through, in order:

Layer	Purpose
helmet	Secure HTTP headers
cors	Only the storefront's origin may call the API, with credentials
json body limit	Rejects oversized payloads (100 kb)
cookie-parser	Reads the refresh-token cookie
rate limiter	Caps requests per minute (stricter on auth routes)
route handlers	auth / products / orders / stats / uploads
error handler	Turns errors into clean JSON; hides internals in production

Routes at a glance

- **auth** - register, login, refresh, logout, current user.
- **products** - public catalogue + staff create/edit/delete.
- **orders** - create (guest or member), my orders, one order, invoice, staff status updates.
- **stats** - dashboard figures for staff.
- **uploads** - product-image upload (staff only).

Services

Invoices are generated on demand as branded PDFs with pdfkit. **Email** uses nodemailer; without mail credentials configured it safely logs messages instead of sending, so development never emails real people.

Section 7

Database Structure

Three MongoDB collections, defined as Mongoose models.

User

Field	Type	Notes
name	String	Required
email	String	Required, unique, indexed
passwordHash	String	bcrypt hash; never returned by the API
role	Enum	customer / admin / super-admin / inventory-manager / ...
addresses	[Address]	Saved delivery addresses
refreshTokenHashes	[String]	Hashed active sessions; never returned
createdAt / updatedAt	Date	Timestamps

Product

Field	Type	Notes
slug	String	Unique URL identifier, auto-generated
name, tagline	String	Display name and one-line hook
price	Number	In INR
category	Enum	bags / briefcases / travel / small-goods
collectionSlug	String	Which collection it belongs to
images	[String]	Image URLs; first is the cover
leather, hardware, lining, dimensions	String	Specification
story, details, care	String / [String]	Editorial copy
stock	Number	Units available; never negative
featured	Boolean	Shown in the home showcase
status	Enum	draft / published / archived

Order

Field	Type	Notes
number	String	Human reference, e.g. AAS-XXXX-XXXX
user	Ref / null	Linked account, or null for a guest
email	String	Buyer's email; links guest orders to an account
items	[Snapshot]	Name, image, unit price, qty - frozen at purchase
amounts	Object	subtotal, shipping, total

Field	Type	Notes
shippingAddress	Object	Name, address, city, PIN, phone
status	Enum	confirmed -> ... -> delivered (full lifecycle)
statusHistory	[Entry]	Every status change with a timestamp
payment	Object	Provider, status, reference

Why orders snapshot their items

An order copies each product's name, image and price at the moment of purchase. If a product is later renamed, repriced or deleted, past orders and their invoices stay exactly as the customer bought them - history never changes underfoot.

Section 8

Authentication & Security

How sign-in works

On login the API issues two tokens. A short-lived **access token** (15 minutes) is returned in the response and held in the browser's memory. A long-lived **refresh token** (30 days) is set as a secure, http-only cookie the browser cannot read from JavaScript. When the access token expires, the client quietly exchanges the refresh cookie for a new pair.

Rotating, single-use refresh tokens

Refresh tokens are stored **hashed** in the database, and each one can be used only once - every refresh issues a new token and invalidates the old. If a stolen token is replayed after it has been rotated, the attempt is rejected. Concurrent refreshes (two browser tabs at once) are handled atomically so they never corrupt a session.

Roles & permissions

Every account has a role. Customer-facing routes require a valid login; staff routes additionally require the right role (admin, super-admin, inventory-manager, and so on). These checks run on the **server** - the browser is never trusted.

Defence in depth

- Passwords hashed with bcrypt (cost factor 12) - never stored in readable form.
- All prices and stock recalculated server-side; the browser cannot set a price.
- Stock decrements atomically at checkout, so it can never go negative or oversell.
- Product edits accept only whitelisted fields (no mass-assignment).
- Rate limiting on all routes, stricter on login and registration.
- Uniform 'email or password incorrect' message - never reveals which emails exist.
- Security headers via helmet; CORS locked to the storefront origin.
- Internal errors are logged server-side and never leaked to users in production.

Section 9

Key Workflows

Placing an order (guest or member)

```
Add to cart -> Checkout -> API validates cart & address
-> server re-prices every line from the database
-> atomically reserves stock (never oversells)
-> creates the order (status: confirmed)
-> sends confirmation email + makes invoice available
```

Guests order with just an email; the order is real and stored against that email.

Claiming guest orders

When someone registers or logs in, any past guest orders placed with that same email are automatically attached to the new account - so their history is never lost.

Order tracking

Each order keeps a full status history. The customer's account page renders this as a timeline - Confirmed, In the making, Packed, Shipped, Out for delivery, Delivered - with the date each stage was reached. Staff advance the status from the admin panel, and the customer sees it immediately.

Managing products

- **Add** - a form creates a draft; the URL slug is generated automatically and kept unique.
- **Edit** - change any field; only whitelisted fields are accepted by the server.
- **Duplicate** - copy a piece as a new draft.
- **Delete** - permanently remove it (past orders keep their own snapshot).
- **Publish / Archive** and **stock** can be changed inline from the list.

Uploading product images

```
Pick image files -> uploaded to the API (staff only, images <= 5 MB)
-> stored with safe random filenames in server/uploads/
-> served back as URLs and attached to the product
```

The storefront (on a different domain) can display them because the image responses carry a cross-origin resource header.

Section 10

The Admin Panel

A private area at /admin, gated by role in the browser and, more importantly, on the server for every action. Non-staff see a polite 'staff only' screen.

Screen	What it does
Dashboard	Revenue, orders, customers, average order value, recent orders, low-stock alerts
Orders	Every order with items and totals; change status through the lifecycle
Inventory	Add / edit / duplicate / delete products; upload images; adjust stock and visibility

Default development login

In development a super-admin is seeded automatically: admin@aasleathers.in / workshop-admin. In production this account is NOT created unless you set ADMIN_EMAIL and ADMIN_PASSWORD - so a known password can never ship live.

Section 11

API Reference

All endpoints are prefixed with /api. 'Auth' = valid login required; 'Staff' = a staff role required.

Method & path	Auth	Purpose
POST /auth/register	-	Create an account
POST /auth/login	-	Sign in
POST /auth/refresh	cookie	Rotate tokens
POST /auth/logout	cookie	End session
GET /auth/me	Auth	Current user
GET /products	-	Published catalogue (filters, sort)
GET /products/:slug	-	One product
POST /products	Staff	Create product
PATCH /products/:slug	Staff	Edit product
DELETE /products/:slug	Staff	Delete (or ?hard=true)
GET /products/admin/list	Staff	Full catalogue incl. drafts
POST /orders	Optional	Place an order (guest or member)
GET /orders/mine	Auth	My orders
GET /orders/:id/invoice	Auth	Download invoice PDF
PATCH /orders/:id/status	Staff	Advance order status
GET /stats	Staff	Dashboard figures
POST /uploads	Staff	Upload product images

Section 12

Running It Locally

Two terminals. No database install needed - the server starts its own in-memory MongoDB.

1. Start the API

```
cd server
npm install
npm start
# -> http://localhost:5000 (seeds catalogue + admin account)
```

2. Start the website

```
cd web
npm install
npm run dev
# -> http://localhost:3000 (talks to the API on :5000)
```

Note

The in-memory development database resets every time the API restarts. That is intended for local work; production uses a persistent MongoDB Atlas cluster.

Section 13

Putting It Online

Designed for a Hostinger Business plan, but the shape applies to any host. Three pieces, deployed once; about an hour end-to-end. The full click-by-click guide is in `DEPLOY.md`.

Step 1 - Database (MongoDB Atlas, free)

- Create a free account and an M0 cluster (choose a Mumbai region).
- Add a database user and allow network access.
- Copy the connection string - this becomes `MONGODB_URI`.

Step 2 - API (Hostinger Node.js app)

- Create an 'api' subdomain (api.yourdomain.com).
- Create a Node.js application; startup file `src/server.js`; upload the `server/` folder.
- Set the environment variables (see next section), install dependencies, and start.
- Check `https://api.yourdomain.com/api/health` returns ok. The catalogue seeds itself.

Step 3 - Website (static upload)

```
cd web
# point the build at your API (optional - it also auto-detects api.<domain>)
npm run build
# upload everything inside the 'out/' folder into public_html, then enable SSL
```

Why the API sits on a subdomain

The login cookie is shared across sub-domains of one site but not across unrelated domains. Hosting the API at `api.yourdomain.com` means sign-in works with no extra configuration. Also ensure the server's `uploads/` folder is preserved across redeloys so product photos are not lost.

Section 14

Environment Variables

Set these on the API server. Generate each JWT secret with a long random string.

Variable	Required	Purpose
<code>NODE_ENV</code>	prod	Set to 'production' when live
<code>MONGODB_URI</code>	prod	MongoDB Atlas connection string

Variable	Required	Purpose
CLIENT_URL	yes	Your storefront origin (for CORS)
JWT_ACCESS_SECRET	yes	Signs access tokens (long random)
JWT_REFRESH_SECRET	yes	Signs refresh tokens (long random)
ACCESS_TOKEN_TTL	no	Access-token lifetime (default 15m)
REFRESH_TOKEN_TTL_DAYS	no	Refresh-token lifetime (default 30)
ADMIN_EMAIL / ADMIN_PASSWORD	prod	Creates the first admin on launch
SMTP_URL	no	Real email sending; logs instead if unset
MAIL_FROM	no	Sender name/address on emails
NEXT_PUBLIC_API_URL	no	Override the auto-detected API URL (frontend)

Production safety guard

The server refuses to start in production if the JWT secrets are still the development defaults, or if MONGODB_URI is missing - preventing an insecure launch by accident.

Section 15

Known Limitations & Roadmap

Current limitations

- **Payments** - Razorpay is not yet wired in; orders record payment as 'pending'. Planned as the final step.
- **New product detail pages** - listings update live, but a brand-new product's own detail page needs a site rebuild to exist on a static host (works instantly in development).
- **Detail-page price** - listing prices update live; the individual product page shows the built-in price until a rebuild (customers are always charged the correct database price).
- **Email verification** - not yet implemented, so guest orders can be claimed by whoever registers that email. On the roadmap before launch.
- **Uploaded images** - stored on the API server (local); no drag-reorder yet, and removed images are not garbage-collected.
- **Legal pages** - contain placeholders (business name, GSTIN, grievance officer) that must be filled with real details and reviewed by a lawyer before going live.

Roadmap

- Email verification.
- Admin modules: categories, collections, coupons, reviews.
- Server-synced wishlist and cart across devices.
- Razorpay payments with webhook verification (final step).
- Optional: move image storage to Cloudinary for automatic optimisation and CDN delivery.

This document reflects the platform as built to date. It is living documentation - as modules such as payments and email verification are added, the relevant sections should be updated alongside them.